

제작보고서

이력서 AI

AI 기반 이력서 작성 및 재취업 지원 웹 서비스

구분	내용
프로젝트명	이력서 AI
프로젝트 유형	React 기반 웹 서비스 / AI 이력서 생성 서비스
핵심 구현	단계형 이력서 입력, GPT API 문장 생성, 회원 인증, MongoDB 저장, 채용공고 추천
작성자	김재현
제출일	2026. 06. 08.

본 보고서는 웹 페이지 내 제작보고서 내용을 기반으로 프로젝트 목적, 페이지 구조, 디자인 방향, 주요 기능, 사용 기술, 핵심 코드 및 AI 활용 과정을 제출용 문서 형식으로 정리한 것이다.

목차

1. 개요
2. 페이지 구조
3. 디자인 컨셉
4. 주요 기능 및 사용 기술
5. 핵심 코드 리뷰
6. 주요 AI 프롬프트 기록
7. 한계점 및 발전 방향
8. 결론

1. 개요

본 프로젝트의 서비스명은 이력서 AI이다. 이 서비스는 이력서 작성 경험이 부족하거나 디지털 문서 작성에 익숙하지 않은 중장년 구직자가 자신의 경력과 강점을 체계적으로 정리할 수 있도록 돕는 AI 기반 이력서 작성 및 재취업 지원 웹 서비스이다.

기존 이력서 작성 서비스는 사용자가 모든 내용을 직접 작성해야 하는 경우가 많아, 경력 표현 방식이나 자기소개서 문장 구성에 어려움을 느끼는 사용자가 접근하기 어렵다. 이에 본 프로젝트는 단계형 입력 방식, 선택형 UI, GPT API 기반 문장 생성, ATS 적합도 분석, 채용공고 추천 기능을 결합하여 이력서 작성부터 지원 준비까지 이어지는 흐름을 구현하였다.

서비스의 개발 범위는 React 기반 화면 구현, Node.js API 서버, MongoDB 데이터 저장, OpenAI API 연동, 회원가입 및 로그인, 세션 관리, 반응형 UI 구성, Vercel 배포 구조 정리까지 포함한다. 단순한 정적 페이지가 아니라 사용자 입력, AI 생성, 데이터 저장, 추천 기능이 연결되는 웹 애플리케이션 형태로 제작하였다.

프로젝트 핵심 정의

이력서 AI는 사용자의 경력·강점·희망 직무를 입력받아 AI 이력서 초안을 생성하고, ATS 적합도와 문장 완성도, 추천 직무 및 채용공고를 함께 제공하는 재취업 지원 서비스이다.

1.1 개발 배경

- 재취업 준비자는 자신의 경험을 어떤 표현으로 정리해야 할지 어려움을 겪는 경우가 많다.
- 중장년 사용자는 복잡한 입력 폼이나 긴 자기소개서 작성 과정에서 서비스 이용 부담을 느낄 수 있다.
- 채용 과정에서는 이력서 문장뿐 아니라 직무 키워드, 경력 정리, 지원 가능 공고 탐색까지 함께 고려해야 한다.
- 따라서 본 프로젝트는 사용자가 긴 문장을 처음부터 작성하지 않아도 되도록 입력 과정을 단계화하고, AI가 초안을 생성하여 작성 부담을 줄이는 방향으로 설계하였다.

1.2 프로젝트 목표

목표	구체적인 구현 방향
작성 부담 완화	한 화면에 많은 항목을 배치하지 않고 5단계 입력 흐름을 제공한다.
개인화된 결과 제공	지원 직무, 경력, 성과, 자격증, 강점, 목표를 조합하여 AI 이력서 초안을 생성한다.
서비스 신뢰성 확보	비밀번호 해시 저장, 세션 토큰 해시화, HttpOnly 쿠키 등 기본 보안 구조를 반영한다.
실제 활용성 강화	생성 결과 저장, 복사, PDF 인쇄, 채용공고 추천 기능을 함께 제공한다.

2. 페이지 구조

프로젝트는 React Router를 활용한 SPA 구조로 구성하였다. 사용자는 공통 Header를 통해 서비스 소개, AI 이력서 작성, 채용공고, 구독, 리뷰, 제작보고서 페이지로 이동할 수 있다. 주요 페이지는 각각 독립적인 컴포넌트로 분리하여 유지보수성과 확장성을 높였다.

경로	페이지	역할	주요 내용
/	홈	서비스 진입	서비스 핵심 가치와 사용 대상을 소개하고 이력서 작성 화면으로 이동시킨다.
/service	서비스 소개	문제 정의	중장년 재취업자의 이력서 작성 어려움과 해결 방식을 설명한다.
/resume	AI 이력서	핵심 기능	5단계 입력값을 수집하고 GPT API를 통해 이력서 초안과 분석 결과를 생성한다.
/jobs	채용공고	지원 연결	생성된 이력서의 직무·경력 키워드와 연결되는 공고 목록을 제공한다.
/subscription	구독	수익 모델	Free, Pro, Team 플랜을 구분하여 서비스 확장 가능성을 제시한다.
/reviews	사용자 리뷰	사용자 참여	로그인 사용자의 리뷰 작성과 리뷰 목록 조회 기능을 제공한다.
/report	제작보고서	프로젝트 문서	제작 목적, 구조, 디자인, 기능, 기술, 코드, AI 활용 과정을 정리한다.

2.1 전체 서비스 흐름

사용자 접속

- > 회원가입 또는 로그인
- > AI 이력서 작성 페이지 이동
- > 기본 정보, 직무, 경력, 성과, 강점, 목표를 5단계로 입력
- > /api/resume 요청
- > 서버에서 OpenAI Responses API 호출
- > 이력서 초안 및 자기소개서 문장 생성
- > ATS 점수, 문장 완성도, 추천 직무 계산
- > 로그인 사용자는 /api/resumes로 결과 저장
- > 채용공고 추천, 복사, PDF 인쇄 기능 활용

그림 1. 이력서 AI의 사용자 서비스 흐름

이 구조는 사용자가 단순히 문장을 생성하는 데서 그치지 않고, 입력 정보 정리, AI 초안 생성, 결과 저장, 채용 공고 탐색까지 이어지는 하나의 재취업 준비 흐름을 경험하도록 설계한 것이다.

2.2 폴더 및 구현 구조

구분	주요 파일 및 폴더	역할
화면	src/pages	Home, Service, ResumeCreate, Jobs, Subscription, Reviews, Report 페이지 구현
공통 컴포넌트	src/components	Header, LoginModal 등 반복 사용되는 UI 요소 구현
프론트 API	src/api.js	fetch 요청, credentials 포함, X-Requested-With 헤더 적용
로컬 서버	server.js	정적 파일 제공, API 처리, 보안 헤더, 인증, OpenAI 호출
Vercel API	api/	resume, login, signup, me, resumes, reviews, jobs, health 엔드포인트 분리
DB 모듈	server/database.js	MongoDB 연결, 컬렉션 생성, 인덱스 생성, 기본 채용 공고 시드
공유 유틸	serverless/	인증, HTTP 응답, 정규화 로직을 서버리스 환경에서 재사용

3. 디자인 컨셉

디자인의 기본 방향은 명확성, 신뢰감, 사용 편의성이다. 본 서비스는 이력서 작성이라는 실용적인 목적을 가진 서비스이므로 화려한 장식보다 사용자가 현재 해야 할 작업을 쉽게 이해할 수 있는 화면 구성을 우선하였다.

디자인 원칙	구현 내용
쉬운 작성 경험	긴 서술형 입력보다 버튼 선택과 짧은 문장 입력을 중심으로 구성하였다. 사용자는 단계별 안내에 따라 필요한 정보만 순서대로 입력한다.
신뢰감 있는 색상	파란색 계열을 주 색상으로 사용하여 안정감과 전문성을 표현하였다. 흰색 카드형 UI를 활용해 입력 영역과 결과 영역을 명확히 구분하였다.
결과 중심 레이아웃	입력 폼, AI 생성 결과, 분석 점수, 추천 직무, 채용공고가 구분되어 사용자가 결과를 빠르게 확인할 수 있다.
반응형 접근성	CSS Grid와 Flexbox, Media Query를 사용하여 데스크톱과 모바일에서 카드와 버튼이 자연스럽게 재배치되도록 하였다.

3.1 사용자 경험 설계

이력서 작성 화면은 한 번에 모든 정보를 요구하지 않고, 기본 정보부터 경력과 목표까지 순차적으로 입력하도록 구성하였다. 사용자는 현재 단계에서 필요한 항목만 확인하므로 입력 부담이 낮아진다. 또한 직무, 경력, 성과, 자격증, 강점 등은 선택 버튼으로 제공하되, 직접 입력 항목도 함께 지원하여 개인의 실제 경험을 반영할 수 있게 하였다.

3.2 시각적 완성도

전체 화면은 카드형 섹션, 일정한 여백, 통일된 버튼 스타일을 사용하여 문서 작성 서비스에 적합한 단정한 분위기를 유지하였다. 보고서 페이지 역시 프로젝트 개요, 페이지 구조, 기술 설명, 코드 리뷰가 카드 단위로 구분되도록 설계하여 평가자가 구현 내용을 빠르게 파악할 수 있도록 하였다.

4. 주요 기능 및 사용 기술

4.1 주요 기능

기능	구현 내용
5단계 이력서 작성	기본 정보, 직무, 경력, 성과, 자격증, 강점, 목표를 단계별로 입력한다.
선택형 입력과 직접 입력 병합	버튼 선택값과 사용자가 직접 입력한 값을 하나의 이력서 데이터로 통합한다.
GPT API 이력서 생성	입력값을 서버로 전달하고 OpenAI Responses API를 통해 이력서 초안을 생성한다.
ATS 및 문장 완성도 분석	직무 관련 키워드, 경력 항목, 강점, 목표 문장을 기준으로 분석 점수를 계산한다.
맞춤 채용공고 추천	사용자의 희망 직무와 경력 키워드를 채용공고 키워드와 비교하여 관련 공고를 추천한다.
회원 인증과 세션 유지	회원가입, 로그인, 로그아웃, 현재 사용자 확인 기능을 제공한다.
이력서 저장 및 조회	로그인 사용자는 생성된 이력서와 분석 점수를 MongoDB에 저장하고 다시 확인할 수 있다.
리뷰 작성 및 조회	로그인 사용자만 후기를 작성할 수 있으며, 전체 리뷰 목록을 조회할 수 있다.
복사 및 PDF 인쇄	생성 결과를 클립보드에 복사하거나 브라우저 인쇄 기능을 통해 PDF로 저장할 수 있다.

4.2 사용 기술

영역	기술	적용 내용
Frontend	React 19, React Router	페이지 컴포넌트 분리, SPA 라우팅, 상태 기반 입력 흐름 구현
Build	Vite 8	빠른 개발 서버, dist 빌드, Vercel 배포용 정적 파일 생성
Styling	CSS Variables, Grid, Flexbox, Media Query	색상 일관성, 카드형 레이아웃, 반응형 화면 구성
Backend	Node.js HTTP API, Vercel Functions	로컬 서버와 배포 서버리스 API를 모두 지원
Database	MongoDB, Index, TTL Index	회원, 세션, 이력서, 리뷰, 채용공고 저장 및 조회 성능 확보
AI	OpenAI Responses API	사용자 입력 기반 이력서 초안 및 자기소개서 생성
Security	crypto, scrypt, SHA-256, HttpOnly Cookie	비밀번호 해시, 세션 토큰 해시화, 쿠키 기반 인증
Browser API	localStorage, Clipboard API, Print API	임시 저장, 결과 복사, PDF 저장 및 인쇄 기능 구현

4.3 데이터베이스 및 보안 설계

MongoDB는 기능별 컬렉션을 분리하여 설계하였다. 회원 정보, 세션 정보, 이력서 결과, 리뷰, 채용공고 데이터를 별도 컬렉션으로 저장함으로써 데이터의 역할을 명확히 하고, 사용자별 조회와 향후 기능 확장을 고려하였다.

컬렉션	주요 필드	역할
users	id, name, email, password_salt, password_hash, created_at	회원 계정 정보를 저장한다. 비밀번호 원문은 저장하지 않고 salt와 scrypt 해시값만 저장한다.
sessions	token_hash, user_id, created_at, expires_at	로그인 세션을 저장한다. expires_at에는 TTL 인덱스를 적용하여 만료 세션을 자동 정리한다.
resumes	id, user_id, target_job, form_data, generated_resume, ats_score, writing_score	사용자별 입력값, 생성 결과, 분석 점수를 저장한다.
reviews	id, user_id, role, rating, text, created_at	로그인 사용자의 리뷰를 저장하고 사용자 정보와 연결해 표시한다.
jobs	id, company, title, location, deadline, category, description	기본 채용공고 데이터를 저장하고 추천 기준으로 활용한다.

보안 측면에서는 비밀번호 해시 저장, 세션 토큰 해시화, HttpOnly 쿠키, SameSite=Lax, 요청 제한, JSON 본문 크기 제한, 보안 헤더 적용, 정적 파일 경로 보호를 구현하였다. 이를 통해 단순 기능 구현을 넘어 실제 서비스 운영을 고려한 기본적인 보안 구조를 반영하였다.

보안 항목	구현 방식
비밀번호 보호	crypto.randomBytes로 salt를 생성하고 crypto.scryptSync로 64바이트 해시를 저장한다.
로그인 검증	입력 비밀번호를 동일한 방식으로 해시한 뒤 crypto.timingSafeEqual로 비교한다.
세션 보호	브라우저에는 랜덤 토큰을 전달하고 DB에는 SHA-256 token_hash만 저장한다.
쿠키 옵션	ireoksseo_session 쿠키에 HttpOnly, SameSite=Lax, Path, Max-Age를 적용한다.
요청 제한	인증, AI 생성, 리뷰 작성 요청에 rate limit을 적용한다.
응답 보호	CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy 등을 설정한다.

5. 핵심 코드 리뷰

핵심 코드 리뷰에서는 사용자의 입력이 AI 생성 결과와 데이터베이스 저장으로 이어지는 주요 흐름을 중심으로 정리하였다. 단순히 화면을 구현한 것이 아니라, 입력 검증, 데이터 정규화, API 호출, 인증 확인, 저장 처리까지 연결되도록 구현하였다.

5.1 React Router 기반 페이지 구성

App.jsx는 전체 서비스의 라우팅을 담당한다. 각 페이지를 독립 컴포넌트로 분리하고 Header와 LoginModal을 공통으로 사용하여 서비스 전반의 구조를 일관되게 유지하였다.

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/resume" element={<ResumeCreate user={user} />} />
  <Route path="/service" element={<Service />} />
  <Route path="/jobs" element={<Jobs />} />
  <Route path="/subscription" element={<Subscription />} />
  <Route path="/reviews" element={<Reviews user={user} onAuthChange={setUser} />} />
  <Route path="/report" element={<Report />} />
</Routes>
```

5.2 단계형 입력 검증과 AI 이력서 생성

ResumeCreate.jsx의 generateResume 함수는 이 프로젝트의 핵심 실행 흐름이다. 사용자가 입력한 값이 조건을 만족하는지 확인한 뒤, 이력서 데이터를 정리하고 AI 생성 API를 호출한다. 로그인 사용자의 경우 결과를 서버에 저장한다.

```
const generateResume = async () => {
  if (isGenerating) return;

  const message = validateStep();
  if (message) {
    alert(message);
    return;
  }

  const nextResult = { ...form };
  const resumeData = buildResumeData(nextResult);
  const resumeAnalysis = calculateAnalysis(resumeData);

  setIsGenerating(true);
  try {
    const aiResult = await generateAiResume(resumeData);
    const generatedResume = aiResult.generatedResume
      || buildGeneratedResumeText(resumeData);

    setResult({ ...nextResult, generatedResume });

    if (user) {
      await createResume({
        targetJob: resumeData.targetJob,
        formData: nextResult,
        generatedResume,
        atsScore: resumeAnalysis.atsScore,
        writingScore: resumeAnalysis.writingScore,
      });
    }
  } finally {
    setIsGenerating(false);
  }
}
```

```
}  
};
```

해당 코드는 입력 검증, AI 요청, 결과 상태 반영, 로그인 사용자 저장을 하나의 흐름으로 연결한다. 사용자가 중복으로 생성 버튼을 누르는 상황을 방지하기 위해 isGenerating 상태도 함께 사용한다.

5.3 선택값과 직접 입력값 통합

중장년 사용자가 버튼 선택만으로 이력서를 작성할 수 있도록 하되, 개인별 특수 경력도 반영할 수 있도록 직접 입력값을 함께 병합하였다.

```
function buildResumeData(form) {  
  const targetJob = form.targetJob === "직접 입력"  
    ? form.targetJobDetail  
    : form.targetJob;  
  
  const education = form.education === "직접 입력"  
    ? form.educationDetail  
    : form.education;  
  
  return {  
    ...form,  
    targetJob,  
    education,  
    careerItems: mergeSelectedAndDetail(  
      form.careerSummary,  
      form.careerSummaryDetail  
    ),  
    achievementItems: mergeSelectedAndDetail(  
      form.achievement,  
      form.achievementDetail  
    ),  
    skillItems: splitText(form.skillSet),  
  };  
}
```

5.4 프론트엔드 API 요청 구조

src/api.js에서는 모든 API 요청에 JSON Content-Type, X-Requested-With 헤더, credentials 옵션을 공통 적용한다. 이를 통해 세션 쿠키가 포함된 요청을 서버에 전달하고, 서버는 허용된 요청인지 확인할 수 있다.

```
export async function apiRequest(path, options = {}) {
  const response = await fetch(path, {
    headers: {
      "Content-Type": "application/json",
      "X-Requested-With": "ireoksseo",
      ...(options.headers || {}),
    },
    credentials: "include",
    ...options,
  });

  const data = await response.json().catch(() => ({}));
  if (!response.ok) {
    return { ok: false, message: data.message };
  }
  return { ok: true, ...data };
}
```

5.5 GPT API 연동 서버 코드

api/resume.js와 server.js에서는 클라이언트에서 받은 formData를 정규화한 뒤 OpenAI Responses API에 전달한다. API 키는 클라이언트 코드에 포함하지 않고 서버 환경변수로 관리하여 노출 위험을 줄였다.

```
const openAiResponse = await fetch(
  "https://api.openai.com/v1/responses",
  {
    method: "POST",
    headers: {
      Authorization: `Bearer ${process.env.OPENAI_API_KEY}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      model: OPENAI_MODEL,
      instructions:
        "You are a careful Korean resume writing consultant.",
      input: buildResumePrompt(resumeInput),
      max_output_tokens: 1600,
    }),
  }
);
```

프롬프트에는 사용자가 입력한 이름, 나이, 연락처, 지원 직무, 경력, 성과, 자격증, 학력, 강점, 보유 역량, 재취업 목표가 구조화되어 들어간다. 또한 사용자가 말하지 않은 회사명, 날짜, 수치를 자격을 임의로 만들지 않도록 제한 문구를 포함하였다.

5.6 이력서 저장 API

api/resumes.js는 로그인 사용자의 user_id를 확인한 뒤, 해당 사용자에게 속한 이력서만 저장하거나 조회한다. 이를 통해 여러 사용자가 서비스를 이용하더라도 자신의 이력서 결과만 확인할 수 있도록 구성하였다.

```
const resume = {
  id: crypto.randomUUID(),
  user_id: user.id,
  title,
  target_job: targetJob,
  form_data: formData,
  generated_resume: generatedResume,
  ats_score: safeScore(body.ats_score ?? body.atsScore),
  writing_score: safeScore(body.writing_score ?? body.writingScore),
}
```

```
recommended_job: recommendedJob,  
created_at: now,  
updated_at: now,  
};  
  
await resumes.insertOne(resume);
```

5.7 인증 및 세션 보안

회원가입과 로그인은 serverless/auth.js 및 server.js에서 처리한다. 비밀번호는 salt 기반 scrypt 해시로 저장하고, 세션 토큰은 데이터베이스에 원문이 아닌 SHA-256 해시값으로 저장한다.

```
export function hashPassword(password, salt = crypto.randomBytes(16).toString("hex")) {  
  const hash = crypto.scryptSync(password, salt, 64).toString("hex");  
  return { salt, hash };  
}  
  
export function hashToken(token) {  
  return crypto.createHash("sha256").update(token).digest("hex");  
}  
  
export function verifyPassword(password, salt, hash) {  
  const expected = Buffer.from(hash, "hex");  
  const candidate = crypto.scryptSync(password, salt, expected.length);  
  return crypto.timingSafeEqual(candidate, expected);  
}
```

6. 주요 AI 프롬프트 기록

본 프로젝트에서는 AI를 단순한 결과 생성 도구로만 사용하지 않고, 서비스 기획, 화면 설계, 코드 구조 개선, 보안 보완 과정의 보조 도구로 활용하였다. 아래 표는 주요 AI 활용 목적과 프로젝트 반영 결과를 정리한 것이다.

활용 단계	주요 프롬프트 요약	프로젝트 반영 결과
서비스 기획	중장년 재취업자를 위한 AI 이력서 작성 웹 서비스의 문제 정의와 핵심 기능을 정리해 달라.	서비스 대상, 개발 배경, 핵심 기능, 사용자 흐름을 구체화하였다.
페이지 구조	React 기반 이력서 작성 서비스의 페이지 구조와 라우팅 방식을 제안해 달라.	/, /resume, /service, /jobs, /subscription, /reviews, /report 구조를 확정하였다.
입력 UX	이력서 작성이 익숙하지 않은 사용자를 위한 단계형 입력 UI를 설계해 달라.	5단계 입력 폼, 선택형 버튼, 직접 입력 병합 구조를 구현하였다.
AI 생성 기능	사용자 입력값을 기반으로 이력서 초안과 자기 소개서를 생성하는 GPT API 프롬프트를 작성해 달라.	buildResumePrompt 함수와 /api/resume 엔드포인트 설계에 반영하였다.
데이터베이스	회원, 세션, 이력서, 리뷰, 채용공고를 MongoDB에 어떻게 나누어 저장하면 좋은지 정리해 달라.	users, sessions, resumes, reviews, jobs 컬렉션과 인덱스 구조를 구성하였다.
보안 개선	비밀번호 해시 저장, 세션 쿠키, 요청 제한, 보안 헤더를 적용하는 방법을 알려 달라.	bcrypt 해시, SHA-256 token_hash, HttpOnly 쿠키, rate limit, 보안 헤더를 적용하였다.

AI 활용에 대한 정리

본 프로젝트에서 AI는 코드를 대신 작성하는 역할에만 한정되지 않았다. 서비스 기획을 구체화하고, 사용자 흐름을 정리하며, 보안 및 데이터베이스 설계 방향을 검토하는 보조 도구로 활용되었다.

7. 한계점 및 발전 방향

본 프로젝트는 AI 이력서 생성과 재취업 지원 흐름을 웹 서비스 형태로 구현하였으나, 실제 운영 서비스로 확장하기 위해서는 다음과 같은 보완이 필요하다.

구분	현재 한계	발전 방향
AI 결과 검증	사용자 입력이 부족할 경우 결과 문장의 구체성이 낮아질 수 있다.	질문형 추가 입력, 예시 문장 추천, 생성 결과 수정 가이드 기능을 추가한다.
채용공고 추천	현재는 기본 공고 데이터와 키워드 기반 추천 중심이다.	공공 채용 API 또는 구인 플랫폼 데이터와 연동하여 실시간 공고 추천으로 확장한다.

구분	현재 한계	발전 방향
데이터 분석	ATS 점수와 문장 완성도는 자체 기준에 기반한 간단한 계산 방식이다.	실제 채용공고 키워드와 이력서 문장 간 유사도 분석, 직무별 가중치 적용을 추가한다.
사용자 접근성	중장년 사용자를 고려했지만 음성 안내나 큰 글자 모드 등은 아직 제한적이다.	고대비 모드, 글자 크기 조절, 단계별 음성 안내를 추가한다.
운영 안정성	로컬 서버와 Vercel Functions 구조를 모두 지원하지만 운영 모니터링은 제한적이다.	로그 수집, 오류 추적, 관리자 페이지, 백업 정책을 추가한다.

8. 결론

이력서 AI는 중장년 재취업자를 주요 사용자로 설정하고, 이력서 작성 과정의 부담을 줄이는 것을 목표로 제작한 AI 기반 웹 서비스이다. 사용자는 단계형 입력 폼을 통해 자신의 경력과 강점을 정리하고, GPT API를 통해 이력서 초안과 자기소개서 문장을 생성할 수 있다.

또한 결과 화면에서 ATS 적합도, 문장 완성도, 추천 직무, 채용공고를 함께 제공함으로써 단순 문장 생성 서비스를 넘어 실제 지원 준비를 돕는 흐름을 구성하였다. 기술적으로는 React, Vite, React Router, Node.js API, MongoDB, OpenAI API, 인증 보안, 반응형 CSS를 통합하여 프론트엔드와 백엔드가 연결된 웹 애플리케이션을 구현하였다.

본 프로젝트는 단순한 GPT 문장 생성 페이지가 아니라, 단계형 사용자 경험, 서버 기반 AI 연동, MongoDB 저장 구조, 해시 기반 인증, 채용공고 추천 기능을 결합한 재취업 지원 서비스라는 점에서 의미가 있다.